CMB FILM SERVICES

Equipment Department

Equipment Inventory Management System Build Plan

Comprehensive Technical Architecture & Implementation Roadmap

Document Version: 1.0

Date: May 28, 2026

Classification: Internal - CMB Equipment Department

Table of Contents

1. Executive Summary

The **CMB Equipment Inventory Management (EIM) System** is a dedicated desktop application designed to serve as the central operations hub for CMB Film Services' equipment department. It will operate as a companion application alongside the existing **CMB Rental Request System** (1 Take), sharing the same Supabase cloud backend for real-time data synchronization.

The EIM system manages the complete lifecycle of equipment assets: from initial acquisition and inventory registration, through rental availability tracking, repair and maintenance workflows, to spare parts and expendables management. Every status change in the EIM system directly affects equipment availability in the main rental app, making it a critical operational dependency for the equipment department.

Key Objectives

- **Centralized Asset Management** -- Single source of truth for all equipment assets, their status, location, and condition.
- **Seamless Supabase Integration** -- Bidirectional real-time sync with the Rental Request System via shared Supabase tables.
- **Equipment Lifecycle Tracking** -- From acquisition to retirement, track every state transition with audit trails.
- **Maintenance Workflow Automation** -- Structured repair/maintenance pipeline with scheduling, parts tracking, and cost analysis.
- **Parts & Expendables Control** -- Inventory management for spare parts, consumables, and expendable items with reorder alerts.
- **Availability Impact Visibility** -- Real-time visibility into how inventory changes affect rental availability across the fleet.

2. System Overview & Architecture

The EIM system follows the same proven Electron-based architecture established by the CMB Rental Request System (1 Take). This ensures consistency in development patterns, deployment mechanisms, and sync behavior across both applications.

High-Level Architecture Diagram

Layer	Component	Description
Cloud	Supabase (PostgreSQL)	Shared cloud database -- single source of truth for equipment catalog, status, and operational data across all CMB applications
Cloud	Supabase Realtime	WebSocket channels for instant cross-app data propagation; both EIM and Rental apps subscribe to the same tables
App Shell	Electron 33 (Main Process)	Node.js runtime hosting SQLite, Supabase client, PDF generation, and all business logic -- renderer never touches data directly
App Shell	Electron (Preload)	Secure bridge via contextBridge exposing typed IPC channels; enforces process isolation between UI and data layers
Frontend	React 18 + TypeScript	Component-driven UI with Zustand state management; communicates exclusively through IPC invoke calls
Local DB	better-sqlite3 (SQLite)	Offline-first local database with WAL journaling; syncs bidirectionally with Supabase on connectivity
Sync	SyncManager + OfflineQueue	Manages online/offline transitions, queues mutations when disconnected, replays on reconnect with conflict resolution

Table 2.1 -- Architecture layers and their responsibilities

System Interaction Model

Flow	Direction	Mechanism	Impact
Equipment Added (EIM)	EIM --> Supabase --> Rental App	Catalog sync push + Realtime	New equipment appears in rental catalog
Equipment Unavailable (EIM)	EIM --> Supabase --> Rental App	Status field update + Realtime	Equipment removed from available inventory
Equipment Rented (Rental)	Rental --> Supabase --> EIM	Operational sync + Realtime	EIM shows equipment as currently deployed
Repair Started (EIM)	EIM --> Supabase --> Rental App	Status + maintenance_status fields	Equipment unavailable until repair complete
Parts Consumed (EIM)	EIM local + Supabase	Parts inventory decrement	Stock levels updated, reorder alerts triggered

Table 2.2 -- Cross-application data flow and impact mapping

3. Core Modules

The EIM system is organized into four interconnected core modules, each managing a distinct aspect of the equipment lifecycle. Together, they form a complete asset management pipeline that directly controls equipment availability in the CMB Rental Request System.

ACQ Equipment Acquisition

Intake & registration of new equipment assets

- New equipment registration form
- Auto-assign equipment codes (EIM-XXXX)
- Category/subcategory classification
- Serial number & asset tag tracking
- Purchase order & vendor linkage
- Initial condition assessment
- Auto-push to Supabase on creation
- Bulk import from CSV/Excel

INV Rental Inventory Mgmt

Overall equipment status & availability control

- Real-time availability dashboard
- Status management (Available/Deployed/Hold)
- Location tracking per equipment
- Deployment history timeline
- Unavailability reason logging
- Bulk status updates
- Integration with rental schedules
- Equipment condition grading

R&M Repair & Maintenance

Complete repair lifecycle from report to resolution

- Maintenance request creation
- Repair ticket workflow (queue/WIP/done)
- Technician assignment & tracking
- Parts consumption per repair
- Repair cost accumulation
- Maintenance scheduling (preventive)
- Equipment history & repair log
- Auto-flag recurring issues

PRT Parts Management

Spare parts, expendables & consumables inventory

- Parts catalog with categories
- Stock level tracking (qty on hand)
- Reorder point alerts
- Parts consumption history
- Vendor & pricing management
- Compatible equipment mapping
- Expendables tracking (consumables)
- Purchase request generation

Figure 3.0 -- Four core modules of the EIM system

3. Core Modules

3.1 Equipment Acquisition Management

The Equipment Acquisition module handles the intake and registration of all newly acquired equipment into the CMB inventory. Once an item is registered here, it is automatically pushed to Supabase and becomes visible in the Rental Request System's equipment catalog.

Acquisition Workflow:

- **Step 1 -- New Equipment Entry:** Operator fills registration form with equipment details (name, brand, model, serial number, category, purchase info).
- **Step 2 -- Code Assignment:** System auto-generates unique equipment code following the existing pattern (e.g., CAM-011, LIT-005) based on category prefix.
- **Step 3 -- Classification:** Equipment is assigned to category > subcategory > sub-subcategory hierarchy matching the existing catalog structure.
- **Step 4 -- Pricing Setup:** Base rental price, pricing type (per_day/per_project/package_rate), and any package associations are configured.
- **Step 5 -- Condition Assessment:** Initial condition grade (A/B/C/D) is recorded with notes and optional photo documentation.
- **Step 6 -- Supabase Sync:** On save, the equipment record is pushed to the cloud via catalog-sync, making it available in the Rental Request System.

Field	Type	Required	Description
equipment_code	TEXT	Yes (auto)	System-generated unique code (category prefix + sequence)
name	TEXT	Yes	Equipment name matching rental catalog naming convention
display_name	TEXT	Yes	Full display name shown in catalog views
category_id / subcategory_id	UUID	Yes	Links to existing category hierarchy (shared with Rental)
serial_number	TEXT	Yes	Manufacturer serial number for asset tracking
asset_tag	TEXT	No	Internal CMB asset tag (physical label ID)
brand / model	TEXT	Yes	Manufacturer and model identification
purchase_date	DATE	Yes	Date of acquisition
purchase_price	NUMERIC	Yes	Acquisition cost for depreciation and ROI tracking
vendor_name	TEXT	No	Supplier/vendor for warranty and reorder reference
warranty_expiry	DATE	No	Warranty end date for maintenance planning
condition_grade	TEXT	Yes	Initial condition (A=Excellent, B=Good, C=Fair, D=Poor)
base_price	NUMERIC	Yes	Default daily rental rate
pricing_type	TEXT	Yes	per_day | per_project | package_rate
notes	TEXT	No	Acquisition notes, included accessories, etc.

Table 3.1 -- Equipment Acquisition registration fields

3.2 Rental Equipment Inventory Management

This module is the operational nerve center for equipment status management. It provides a real-time view of every asset's current state and directly controls whether equipment appears as available for rent in the main application. When an equipment item becomes unavailable -- whether due to repair, hold, deployment, or retirement -- the status change is made here and propagated to Supabase.

Status	Code	Color	Rental Impact	Description
Available	AVAILABLE	Green	Rentable	Equipment is in warehouse, ready for deployment
Deployed	DEPLOYED	Blue	Not Rentable	Currently out on a rental project
In Repair	IN_REPAIR	Orange	Not Rentable	Undergoing repair/maintenance in workshop
On Hold	ON_HOLD	Yellow	Not Rentable	Reserved or held for specific upcoming project
In Transit	IN_TRANSIT	Cyan	Not Rentable	Being transported between locations
Retired	RETIRED	Gray	Permanently Removed	End of life -- removed from active inventory
Missing	MISSING	Red	Not Rentable	Unaccounted for -- investigation required
For Inspection	FOR_INSPECTION	Purple	Not Rentable	Returned from rental, pending condition check

Table 3.2 -- Equipment status definitions and rental impact

Key Features:

- **Status Dashboard:** Grid/list view of all equipment with color-coded status badges, filterable by category, status, and location.
- **Quick Status Toggle:** One-click status changes with mandatory reason logging for audit trail compliance.
- **Location Tracking:** Current physical location (Warehouse, Project Site, Workshop, In Transit) with optional GPS/address.
- **Deployment History:** Complete timeline showing where each item has been, when, and for which project.
- **Batch Operations:** Select multiple items for bulk status changes (e.g., mark all returned items as "For Inspection").
- **Availability Calendar:** Visual timeline showing equipment commitments and projected availability windows.
- **Condition Tracking:** Post-rental condition assessments that feed into the maintenance scheduling pipeline.
- **Supabase Sync Impact:** Every status change triggers an immediate Supabase update that the Rental App receives via Realtime.

3.3 Repair & Maintenance Management

When equipment is flagged as needing repair or maintenance -- whether from a post-rental inspection, a technician report, or a scheduled maintenance trigger -- this module manages the entire workflow from initial report through to resolution and return to active inventory.

Repair Workflow Pipeline:

Stage	Status	Actions	Auto-Triggers
1. Reported	REPORTED	Log issue description, severity, reporter name, attach photos	Equipment status --> FOR_INSPECTION Notification to maintenance lead
2. Assessed	ASSESSED	Diagnose root cause, estimate repair cost, list required parts	Parts availability check Cost estimate generated
3. Queued	QUEUED	Assign priority (Critical/High/ Medium/Low), schedule slot	Equipment status --> IN_REPAIR Calendar entry created
4. In Progress	IN_PROGRESS	Technician working, log time, consume parts from inventory	Parts stock decremented Repair timer running
5. Testing	TESTING	Post-repair quality check, functional verification	Test checklist generated based on equipment type
6. Completed	COMPLETED	Final sign-off, update condition grade, close ticket	Equipment status --> AVAILABLE Cost summary finalized
7. Escalated	ESCALATED	External vendor repair needed, ship-out tracking	Equipment status remains IN_REPAIR Vendor PO generated

Table 3.3 -- Repair workflow pipeline stages

Maintenance Types:

- **Corrective Maintenance:** Reactive repairs triggered by equipment failure or damage reports.
- **Preventive Maintenance:** Scheduled maintenance based on usage hours, rental count, or calendar intervals.
- **Predictive Maintenance:** Data-driven alerts based on repair history patterns (e.g., equipment with 3+ repairs in 6 months).

Field	Type	Description
ticket_number	TEXT	Auto-generated repair ticket ID (RPR-YYYY-XXXX)
equipment_id	UUID	FK to equipment_items -- the item being repaired
reported_by	TEXT	Person who reported the issue
reported_date	DATE	Date issue was first reported
issue_description	TEXT	Detailed description of the problem
severity	TEXT	CRITICAL | HIGH | MEDIUM | LOW
repair_status	TEXT	Current pipeline stage (REPORTED through COMPLETED)
assigned_technician	TEXT	Technician responsible for the repair
diagnosis	TEXT	Root cause analysis notes
estimated_cost	NUMERIC	Projected repair cost (parts + labor)
actual_cost	NUMERIC	Final repair cost after completion

parts_consumed	JSON	Array of {part_id, qty, cost} used in repair
labor_hours	NUMERIC	Total technician hours spent
completion_date	DATE	Date repair was completed and equipment returned
post_repair_grade	TEXT	Condition grade after repair (A/B/C/D)

Table 3.4 -- Repair ticket data model

3.4 Parts Management

The Parts Management module tracks all spare parts, expendable supplies, and consumable items used by the equipment department. It maintains stock levels, triggers reorder alerts, maps parts to compatible equipment, and records consumption history for cost analysis.

Parts Categories:

Category	Description	Examples	Tracking Method
Spare Parts	Replacement components for equipment repair	Bulbs, lenses, gears, motors, fans, power supplies	Per-unit with serial tracking for high-value parts
Expendables	Single-use items consumed during production	Gels, diffusion, tape, gaffer tape, batteries (disposable)	Quantity-based batch tracking with lot numbers
Consumables	Items that deplete with use and need regular restocking	Cleaning supplies, lubricants, cable ties, heat shrink	Quantity-based with auto-reorder thresholds
Accessories	Add-on items that accompany equipment rentals	Cables, adapters, mounting hardware, cases	Per-unit tracking linked to parent equipment

Table 3.5 -- Parts category definitions

Key Features:

- **Parts Catalog:** Searchable inventory with category filters, compatible equipment mapping, and vendor information.
- **Stock Level Dashboard:** Visual indicators (green/yellow/red) for current stock vs. minimum thresholds.
- **Consumption Tracking:** Automatic decrement when parts are consumed during repairs; full consumption history log.
- **Reorder Alerts:** Configurable minimum stock thresholds that trigger visual alerts and optional notification.
- **Vendor Management:** Preferred vendors per part, pricing history, and lead time tracking.
- **Cost Analysis:** Parts cost per equipment item, per repair, and aggregate departmental spend reports.
- **Compatible Equipment Map:** Each part linked to one or more equipment items for fast lookup during repairs.
- **Inventory Adjustments:** Manual stock corrections with reason codes (damage, shrinkage, audit correction, received).

4. Database Schema Design

The EIM system extends the existing CMB database schema. Tables shared with the Rental Request System (categories, subcategories, equipment_items, package_definitions, package_items) remain unchanged to maintain sync compatibility. New EIM-specific tables are added for asset tracking, maintenance, and parts.

Shared Tables (Existing -- No Modifications)

Table	Owner	Sync Direction	Purpose in EIM
categories	Shared	Bidirectional	Equipment classification hierarchy (top level)
subcategories	Shared	Bidirectional	Equipment classification (second level)
equipment_items	Shared	Bidirectional	Core equipment catalog -- EIM adds items, Rental reads them
package_definitions	Shared	Bidirectional	Equipment packages -- can be created from either app
package_items	Shared	Bidirectional	Package component mappings
users	Shared	Bidirectional	User accounts -- EIM adds inventory/maintenance roles

Table 4.1 -- Shared tables with Rental Request System

New EIM Tables

Table	Purpose	Key Fields
equipment_assets	Extended asset data beyond the rental catalog	equipment_id (FK), serial_number, asset_tag, purchase_date, purchase_price, vendor, warranty_expiry, condition_grade, current_location, current_status, last_inspection_date
asset_status_log	Audit trail of every status change for each asset	asset_id (FK), previous_status, new_status, changed_by, changed_at, reason, related_ticket_id
maintenance_tickets	Repair and maintenance work orders	ticket_number, equipment_id (FK), reported_by, severity, repair_status, assigned_technician, diagnosis, estimated_cost, actual_cost, labor_hours, completion_date
maintenance_notes	Time-stamped notes and updates on repair tickets	ticket_id (FK), author, note_text, note_type (update/ escalation/resolution), created_at
parts_catalog	Master list of all parts, spares, and expendables	part_code, name, category (spare/expendable/consumable/ accessory), unit_of_measure, unit_cost, vendor_name
parts_inventory	Current stock levels and thresholds	part_id (FK), qty_on_hand, qty_reserved, reorder_point, reorder_qty, location, last_count_date
parts_transactions	All stock movements (in/out/adjust)	part_id (FK), transaction_type (receive/consume/adjust/ return), quantity, reference_id, performed_by, notes
parts_compatibility	Maps parts to compatible equipment items	part_id (FK), equipment_id (FK), notes
preventive_schedules	Recurring maintenance schedule definitions	equipment_id (FK), schedule_type (calendar/usage), interval_days, interval_rentals, next_due_date, last_performed
vendors	Vendor/supplier master record	name, contact_person, phone, email, address, payment_terms, notes, is_active

Table 4.2 -- New tables introduced by the EIM system

5. Supabase Sync Strategy

The sync strategy is designed to maintain consistency between the EIM system, the Rental Request System, and the Supabase cloud database. It builds upon the proven sync patterns already operational in the Rental app (SyncManager, catalog-sync, operational-sync, offline-queue).

Sync Architecture

Table Group	Sync Type	Strategy	Conflict Resolution
Catalog Tables (categories, subcategories, equipment_items, packages)	Bidirectional Catalog Sync	Full table pull on connect; push on local mutation; Realtime subscription	Version field comparison; higher version wins; timestamp tiebreaker
Equipment Assets (equipment_assets, asset_status_log)	Bidirectional Operational Sync	Push status changes immediately; pull deployment status from Rental app via Realtime	Last-write-wins with audit trail preservation; status_log is append-only
Maintenance Tables (maintenance_tickets, maintenance_notes)	EIM-Primary Push Sync	EIM is authoritative; pushes to Supabase; Rental reads via Realtime	EIM always wins; read-only from Rental perspective
Parts Tables (parts_catalog, inventory, transactions, compatibility)	EIM-Only Local + Cloud	EIM manages exclusively; syncs to Supabase for reporting/backup	No conflict -- single writer (EIM only)
Vendors & Schedules (vendors, preventive_schedules)	EIM-Only Local + Cloud	EIM manages exclusively; syncs to Supabase for backup purposes	No conflict -- single writer (EIM only)

Table 5.1 -- Sync strategy per table group

Realtime Subscription Channels

The EIM system subscribes to Supabase Realtime channels for all synced tables. When the Rental app changes an equipment item's deployment status (e.g., equipment is rented out), the EIM system receives the update in real-time and reflects it in the inventory dashboard. Conversely, when EIM marks equipment as unavailable, the Rental app receives the update instantly.

Offline Queue Behavior:

- All mutations are executed against local SQLite immediately (offline-first).
- If Supabase is unreachable, mutations are queued in the offline_queue table.
- On reconnection, SyncManager replays queued operations in FIFO order.
- Failed replays are retried with exponential backoff (max 3 attempts).
- Permanently failed items are flagged for manual resolution.

6. Application Structure & UI Layout

The EIM system follows the same UI architecture as the Rental Request System: a sidebar navigation layout with role-based menu visibility, consistent styling via TailwindCSS, and Zustand-powered state management.

Page Structure

Page	Route	Module	Description
Login	/login	Auth	Authentication with role-based redirect
Dashboard	/	Core	Overview: fleet health, alerts, status summary, recent activity feed
Equipment Catalog	/equipment	ACQ + INV	Full equipment list with status, filters, search, and bulk actions
Add Equipment	/equipment/new	ACQ	New equipment registration form with category selection and pricing setup
Equipment Detail	/equipment/:id	INV	Single equipment view: status, history, maintenance log, deployments, condition
Maintenance Queue	/maintenance	R&M	Kanban board of all repair tickets organized by pipeline stage
New Repair Ticket	/maintenance/new	R&M	Create repair/maintenance request with equipment lookup and severity
Ticket Detail	/maintenance/:id	R&M	Full repair ticket view: timeline, notes, parts consumed, cost summary
Parts Inventory	/parts	PRT	Parts catalog with stock levels, alerts, and vendor info
Parts Detail	/parts/:id	PRT	Single part view: stock history, compatible equipment, transactions
Stock Adjustment	/parts/adjust	PRT	Manual stock adjustments with reason codes and approval
Vendors	/vendors	PRT	Vendor management: contacts, payment terms, part associations
Reports	/reports	Core	Analytics: fleet utilization, repair costs, parts spend, availability trends
Settings	/settings	Core	Supabase config, user management, sync status, system preferences

Table 6.1 -- Application pages and routing

Folder Structure

```
src/
main/
database/ # SQLite schema + migrations (extends rental schema)
ipc/ # Domain-split IPC handlers
```

```
equipment.handlers.ts # Equipment CRUD + status management
maintenance.handlers.ts # Repair ticket lifecycle
parts.handlers.ts # Parts inventory operations
vendors.handlers.ts # Vendor management
reports.handlers.ts # Analytics + reporting queries
sync.handlers.ts # Supabase connection + sync controls
auth.handlers.ts # Authentication
sync/ # SyncManager, catalog-sync, offline-queue
pdf/ # Report PDF generation
preload/ # contextBridge IPC surface
renderer/
components/
equipment/ # EquipmentTable, EquipmentForm, StatusBadge
maintenance/ # TicketBoard, TicketForm, RepairTimeline
parts/ # PartsGrid, StockLevel, TransactionLog
common/ # Shared UI components
layout/ # Sidebar, Header, PageWrapper
pages/ # One page component per route
stores/ # Zustand stores (equipment, maintenance, parts, sync)
hooks/ # Custom React hooks
lib/ # IPC wrapper, formatters, helpers
shared/
types/ # IpcChannels + domain interfaces
schemas/ # Zod validation schemas
constants/ # Status enums, category prefixes
database/
schema.sql # SQLite source of truth (extends rental schema)
supabase-migration.sql # PostgreSQL mirror for Supabase
```

7. Tech Stack

The technology stack mirrors the Rental Request System to ensure cross-team consistency, shared development patterns, and minimal onboarding overhead.

Layer	Technology	Version	Purpose
Shell	Electron	33.x	Desktop application shell with main/preload/renderer process isolation
UI Framework	React	18.x	Component-based UI with hooks and functional components
Language	TypeScript	5.3+	Type safety across all processes (main, preload, renderer)
Bundler	Vite	5.x	Fast HMR dev server for renderer; production build optimization
State Management	Zustand	4.5+	Lightweight per-domain stores with devtools support
Styling	TailwindCSS	3.4+	Utility-first CSS matching Rental app design system
Validation	Zod	4.x	Runtime schema validation for IPC payloads and form inputs
Local Database	better-sqlite3	12.x	Synchronous SQLite with WAL journaling (main process only)
Cloud Database	Supabase	2.x	PostgreSQL with Realtime, shared with Rental app
PDF Generation	pdfkit	0.15+	Report and label generation (main process only)
Spreadsheets	ExcelJS	4.x	Import/export of equipment data and inventory reports
Routing	react-router-dom	6.x	Client-side routing for SPA navigation
Icons	lucide-react	0.330+	Consistent icon set matching Rental app
Date Handling	date-fns	3.x	Date formatting and manipulation
IDs	uuid	9.x	UUID v4 generation for all records
Packager	electron-builder	26.x	DMG/ZIP (macOS), NSIS/portable (Windows) builds

Table 7.1 -- Complete technology stack

8. Data Flow & Integration Logic

This section details the critical data flows that connect the EIM system with the main Rental Request System. These integration points are the backbone of operational consistency.

8.1 Equipment Acquisition Flow

Step	Actor	System	Action	Data Impact
1	Operator	EIM	Fill equipment registration form	New record created in local SQLite
2	System	EIM	Validate input via Zod schema	Validation errors shown or proceed
3	System	EIM	Auto-generate equipment_code	Code assigned based on category prefix
4	System	EIM	Insert into equipment_items + equipment_assets	Local DB updated atomically
5	System	EIM	Push to Supabase via catalog-sync	Cloud equipment_items table updated
6	System	Supabase	Broadcast Realtime event	postgres_changes event published
7	System	Rental App	Receive Realtime payload	Upsert into local SQLite via catalog-sync
8	System	Rental App	Refresh equipment store	New equipment appears in rental catalog

Table 8.1 -- Equipment acquisition data flow

8.2 Status Change Impact Flow

Step	Actor	System	Action	Data Impact
1	Operator	EIM	Change equipment status (e.g., Available --> In Repair)	Local equipment_assets.current_status updated
2	System	EIM	Log status change to asset_status_log	Append-only audit record created
3	System	EIM	Update equipment_items.is_active if status is Retired	is_active = 0 removes from rental catalog
4	System	EIM	Push changes to Supabase	Cloud tables updated
5	System	Supabase	Broadcast Realtime events	Both EIM and Rental app receive updates
6	System	Rental App	Update local catalog + refresh UI	Equipment availability reflects new status

Table 8.2 -- Status change propagation flow

8.3 Repair-to-Parts Consumption Flow

Step	Actor	Action	Data Impact
1	Technician	Selects parts needed from parts catalog	Parts reserved (qty_reserved incremented)
2	Technician	Confirms parts consumed during repair	parts_transactions record created (type=consume)

3	System	Decrement parts_inventory.qty_on_hand	Stock level reduced
4	System	Check qty_on_hand vs reorder_point	If below threshold: alert generated
5	System	Add cost to maintenance_tickets.actual_cost	Repair cost accumulation updated
6	System	Sync parts_inventory to Supabase	Cloud inventory levels updated for reporting

Table 8.3 -- Repair parts consumption flow

9. User Roles & Permissions

The EIM system introduces new roles specific to equipment department operations while maintaining compatibility with the existing user table shared via Supabase.

Role	Scope	Permissions
admin	Full System	All operations: user management, settings, data purge, sync configuration, all module access
inventory_manager	ACQ + INV	Add/edit equipment, manage status, view maintenance, view parts, run inventory reports
maintenance_lead	R&M + PRT	Create/manage repair tickets, assign technicians, consume parts, view equipment status
technician	R&M (Limited)	View assigned tickets, log work notes, record parts consumed, update repair status (own tickets only)
parts_clerk	PRT	Manage parts catalog, adjust stock, process receiving, manage vendors, generate purchase requests
viewer	Read-Only	View all dashboards and reports; cannot create, edit, or delete any records

Table 9.1 -- EIM system roles and permissions

Note: The users table role CHECK constraint will be updated in the Supabase migration to include the new EIM-specific roles. The Rental Request System will ignore roles it does not recognize, ensuring backward compatibility.

10. Development Phases & Timeline

The build is structured into six sequential phases, each delivering a functional increment that can be tested and validated before proceeding.

Phase	Name	Scope	Deliverables	Est.
Phase 1	Foundation & Project Setup	Electron app scaffold, DB schema, Supabase migrations, auth	Working app shell with login, sidebar navigation, settings page with Supabase config, initial DB migrations applied to both SQLite and Supabase	1-2 weeks
Phase 2	Equipment Acquisition	Equipment registration, catalog management, category browser	Equipment add/edit forms, auto-code generation, category/subcategory management, CSV import, catalog sync to Supabase (visible in Rental app)	2-3 weeks
Phase 3	Inventory Management	Status tracking, location, availability dashboard, deployment history	Equipment status dashboard with color-coded badges, status change with audit logging, deployment history timeline, batch status operations, Realtime sync	2-3 weeks
Phase 4	Repair & Maintenance	Repair ticket lifecycle, Kanban board, technician assignment, scheduling	Maintenance queue (Kanban view), ticket creation and lifecycle management, technician assignment, notes/timeline, preventive schedule configuration	3-4 weeks
Phase 5	Parts Management	Parts catalog, stock tracking, consumption, vendor management	Parts CRUD with stock levels, consumption during repairs (linked to tickets), reorder alerts, vendor management, inventory adjustments with reason codes	2-3 weeks
Phase 6	Reports, Polish & Deployment	Analytics, PDF reports, UI polish, packaging, final testing	Fleet utilization reports, repair cost analysis, parts spend reports, PDF generation, cross-app integration testing, electron-builder packaging	2-3 weeks

Table 10.1 -- Development phases and estimated timeline (12-18 weeks total)

Phase Dependencies

- **Phase 1 --> Phase 2:** Foundation must be complete before equipment data can be created.
- **Phase 2 --> Phase 3:** Equipment records must exist before status management makes sense.
- **Phase 3 --> Phase 4:** Status system feeds into repair workflow triggers.
- **Phase 2 + Phase 4 --> Phase 5:** Parts are consumed during repairs and linked to equipment.
- **All Phases --> Phase 6:** Reports aggregate data from all modules.

11. Risk Assessment & Mitigation

Risk	Severity	Likelihood	Mitigation Strategy
Sync conflicts between EIM and Rental apps modifying same records	HIGH	MEDIUM	Version-based conflict resolution (higher version wins). Clear ownership model: EIM owns asset data, Rental owns booking data. Append-only audit logs prevent data loss.
Offline data divergence when both apps modify equipment while disconnected	HIGH	LOW	Offline queue with deterministic replay order. Status changes are timestamped; last-write-wins with full audit trail. Manual conflict resolution UI for edge cases.
Database schema drift between EIM and Rental app versions	MEDIUM	MEDIUM	Shared supabase-migration.sql as single PostgreSQL schema source. Migration versioning via schema_migrations table. Backward-compatible schema changes only (additive).
Equipment status inconsistency across applications	HIGH	LOW	Single source of truth in Supabase. Both apps subscribe to Realtime changes. Periodic full-sync reconciliation on app startup (syncOnStartup pattern from Rental app).
Parts stock inaccuracy due to concurrent consumption	MEDIUM	LOW	Atomic transactions for stock decrement. Optimistic locking with version check. Periodic physical inventory audit workflow built into the parts module.
Performance degradation with large equipment and parts catalogs	LOW	MEDIUM	SQLite WAL mode + indexing strategy. Paginated queries. Virtual scrolling for large lists. Selective Supabase syncs (changed records only, not full table pulls).

Table 11.1 -- Risk assessment matrix

12. Success Criteria & Acceptance

The following criteria define what constitutes a successful build of the EIM system. Each criterion must be validated before the system is considered production-ready.

#	Criterion	Validation Method	Pass Condition
C1	Equipment added in EIM appears in Rental app within 5 seconds	Add equipment in EIM; verify in Rental app	Equipment visible in Rental catalog with correct details
C2	Status change in EIM immediately affects rental availability	Mark equipment as IN_REPAIR; check Rental app availability	Equipment not bookable in Rental until status restored
C3	Repair workflow completes full pipeline without data loss	Create ticket, progress through all stages, consume parts	Ticket completes; parts stock accurate; costs tallied
C4	Parts consumption accurately decrements inventory	Consume parts during repair; verify stock levels	qty_on_hand reduced by exact amount; transaction logged
C5	Offline operation queues changes and replays on reconnect	Disconnect Supabase; make changes; reconnect and verify sync	All queued changes applied; no data loss or duplicates
C6	All data survives app restart with no corruption	Perform operations; restart app; verify data integrity	All records intact; sync state preserved
C7	Role-based access correctly restricts operations	Login as each role; attempt operations outside scope	Unauthorized operations blocked with clear messaging
C8	Reports accurately reflect current system state	Generate reports; cross-reference with raw data	Report figures match actual database values exactly

Table 12.1 -- Acceptance criteria checklist

-- End of Document --

CMB Equipment Inventory Management System Build Plan v1.0 | May 28, 2026

CMB Film Services, Inc. | Equipment Department